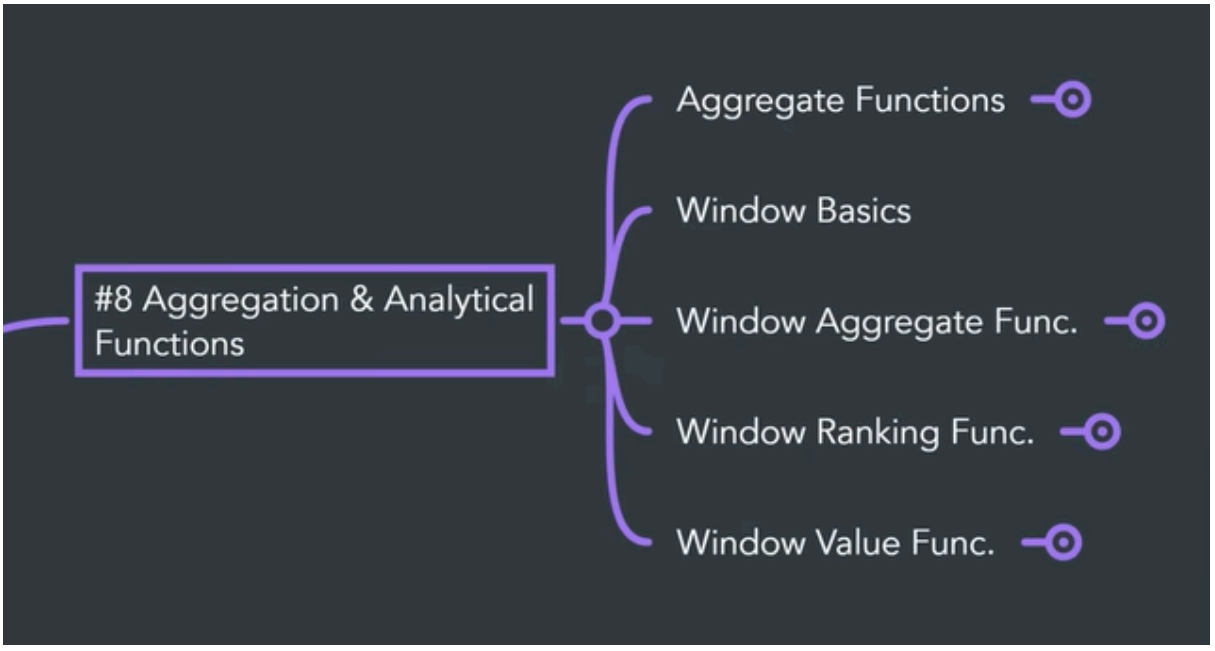
Aggregation & Analytical Functions



1. Aggregate Functions

1. These aggregate functions are used to get the single o/p from combined data
2. These are mostly used with GROUP_BY

```
SELECT
customer_id,
COUNT(*) AS total_nr_orders,
SUM(sales) AS total_sales,
AVG(sales) AS avg_sales,
MAX(sales) AS highest_sales,
MIN(sales) AS lowest_sales
FROM orders
GROUP BY customer_id
```

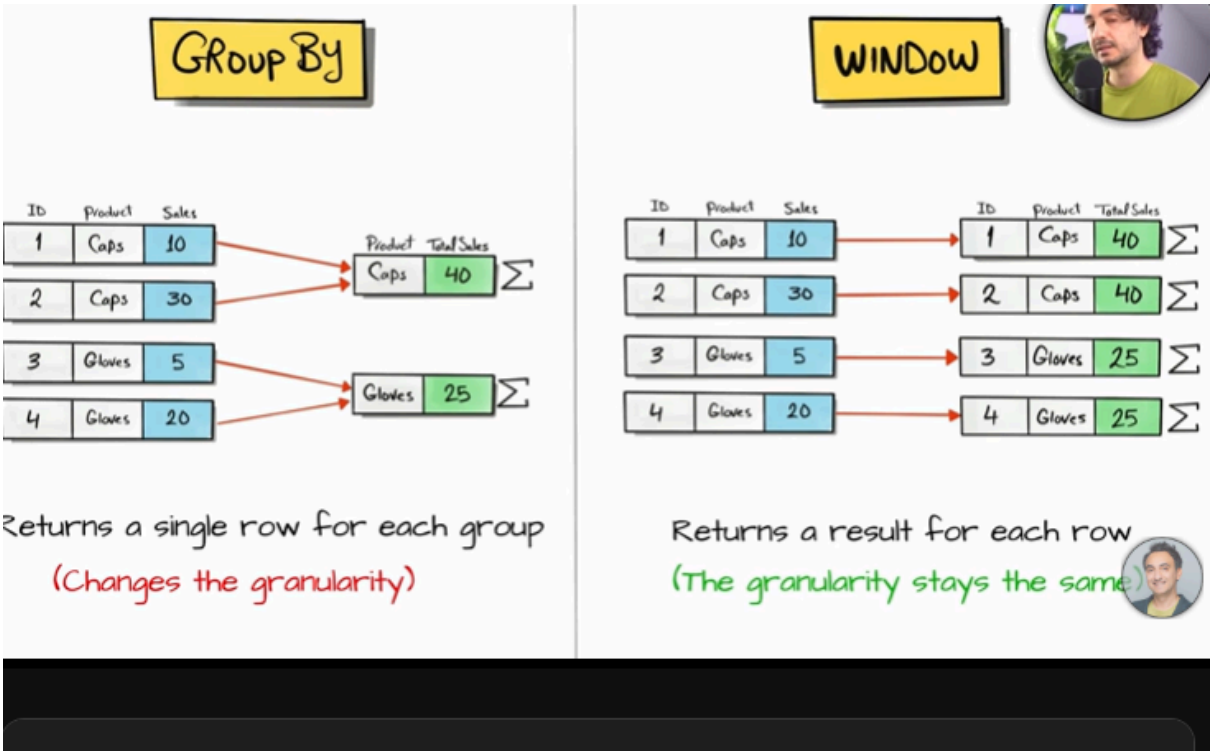
Aggregation Function with Group-By

```sql
SELECT id,
    MAX(CASE WHEN month ='Jan' THEN revenue END) AS Jan_Revenue,
    MAX(CASE WHEN month ='Feb' THEN revenue END) AS Feb_Revenue,
    MAX(CASE WHEN month ='Mar' THEN revenue END) AS Mar_Revenue,
    MAX(CASE WHEN month ='Apr' THEN revenue END) AS Apr_Revenue,
    MAX(CASE WHEN month ='May' THEN revenue END) AS May_Revenue,
    MAX(CASE WHEN month ='Jun' THEN revenue END) AS Jun_Revenue,
    MAX(CASE WHEN month ='Jul' THEN revenue END) AS Jul_Revenue,
    MAX(CASE WHEN month ='Aug' THEN revenue END) AS Aug_Revenue,
    MAX(CASE WHEN month ='Sep' THEN revenue END) AS Sep_Revenue,
    MAX(CASE WHEN month ='Oct' THEN revenue END) AS Oct_Revenue,
    MAX(CASE WHEN month ='Nov' THEN revenue END) AS Nov_Revenue,
    MAX(CASE WHEN month ='Dec' THEN revenue END) AS Dec_Revenue
FROM DEPARTMENT
GROUP BY id
```

we know when we do group-by then all the items with same mentioned column will be aggregated , so for those aggregated data we can use case functions such that for each aggregated row , it was applied seperately

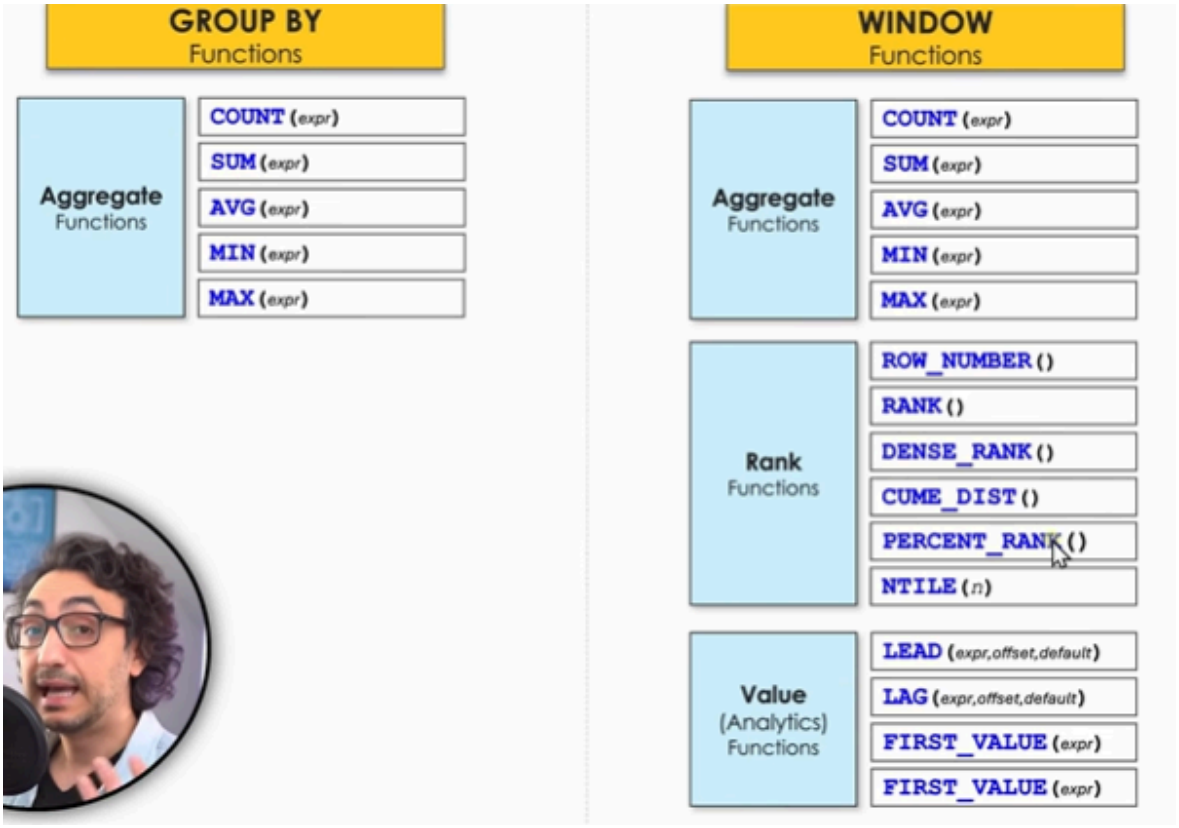
2. Window Function VS Aggregate

1. Performs calculations (eg: aggregation) on a specific subset of data, without losing the level of details rows



2. GROUP_BY ⇒ simple aggregations ,

   WINDOW ⇒ Aggregations+keep details

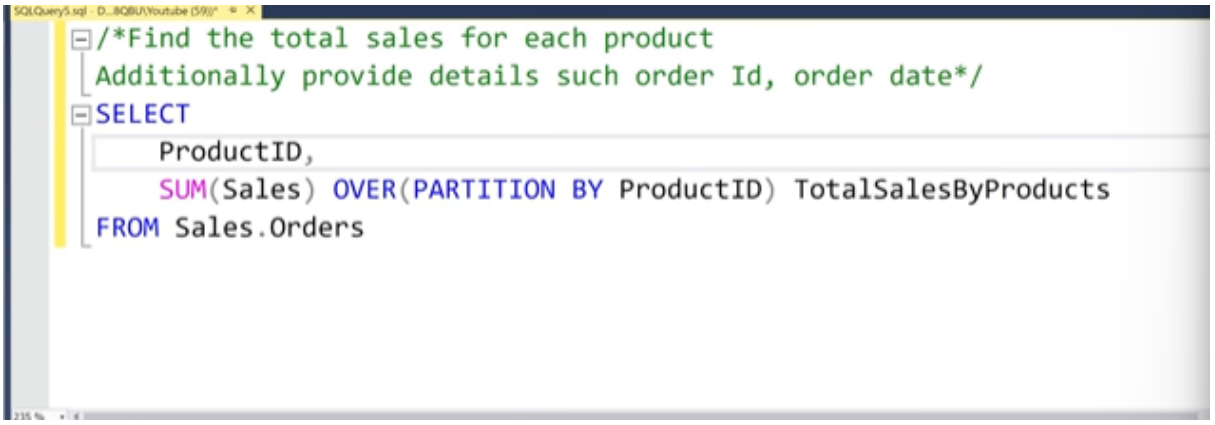


3. Window Basics

1. when we use aggregation functions, with group-by then , only the column written in group-by , column written in Aggregation function are only able to return
2. But to get whole data which are even not included in group-by,Aggregation function in Window functions
3. *** Cant to aggregations and provide the details at the same time, in GROUP_BY , but can be done with WINDOW functions

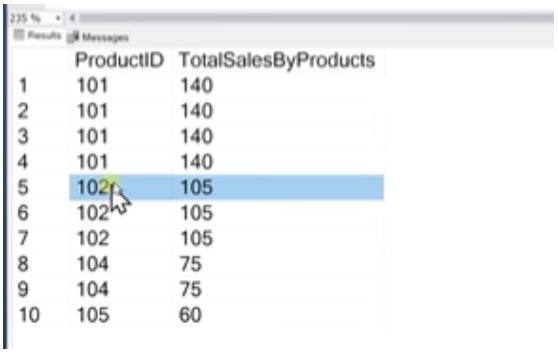


4. here above we have OVER() → this is the keyword says to sql that it was a window function
5. AS we not written anything inside the OVER() that whole data is considered as a single window



```sql
/*Find the total sales for each product
Additionally provide details such order Id, order date*/
SELECT
    ProductID,
    SUM(Sales) OVER(PARTITION BY ProductID) TotalSalesByProducts
FROM Sales.Orders
```

1. Here we wrote Parition by productid , means the data will be divided into windows where each window have the same productid ⇒ Liking grouping the same id's to one window
2. So SUM will sum the data sales of windows data seperately
3. Upto this GROUP_BY will also do . bu main difference is that here we can get every row too



	ProductID	TotalSalesByProducts
1	101	140
2	101	140
3	101	140
4	101	140
5	102	105
6	102	105
7	102	105
8	104	75
9	104	75
10	105	60

so here we can call any column even they not included in the Window function/grouping

4. But in GROUP-BY we get

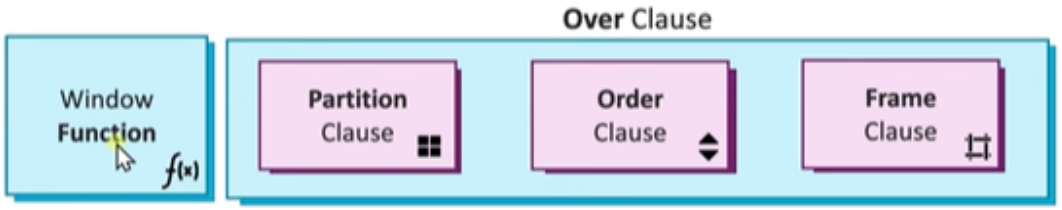
   101 - 140

   102 - 105

   104 - 75

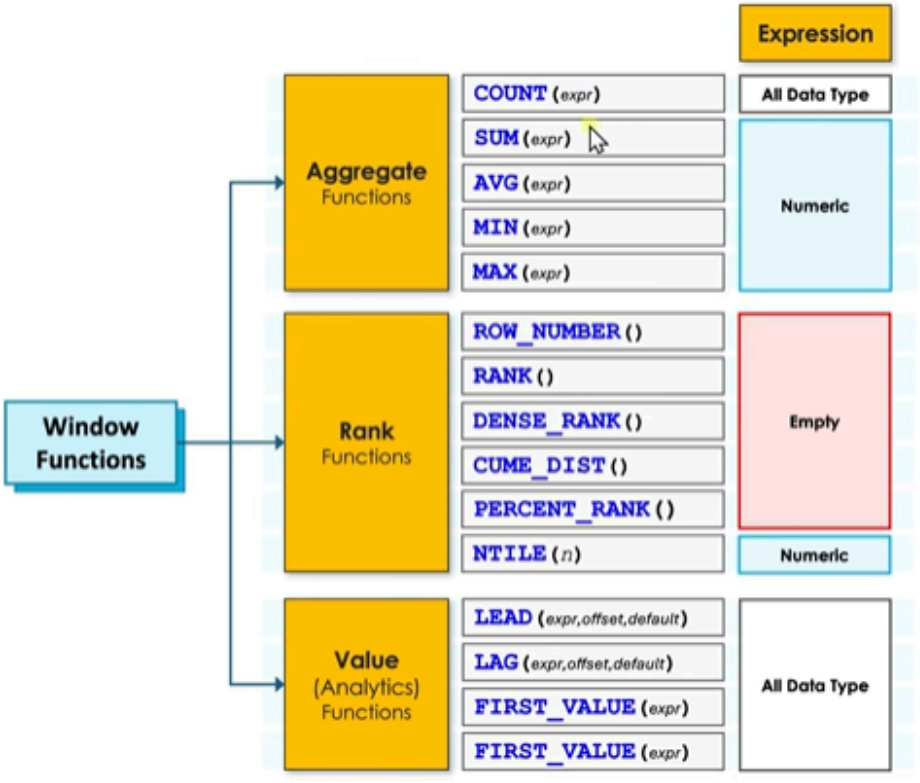
105 - 60

1. Window Syntax



1. WIndow Function → sum,avg,....
2. inside OVER we can write Partition ,Order,Frame clauses

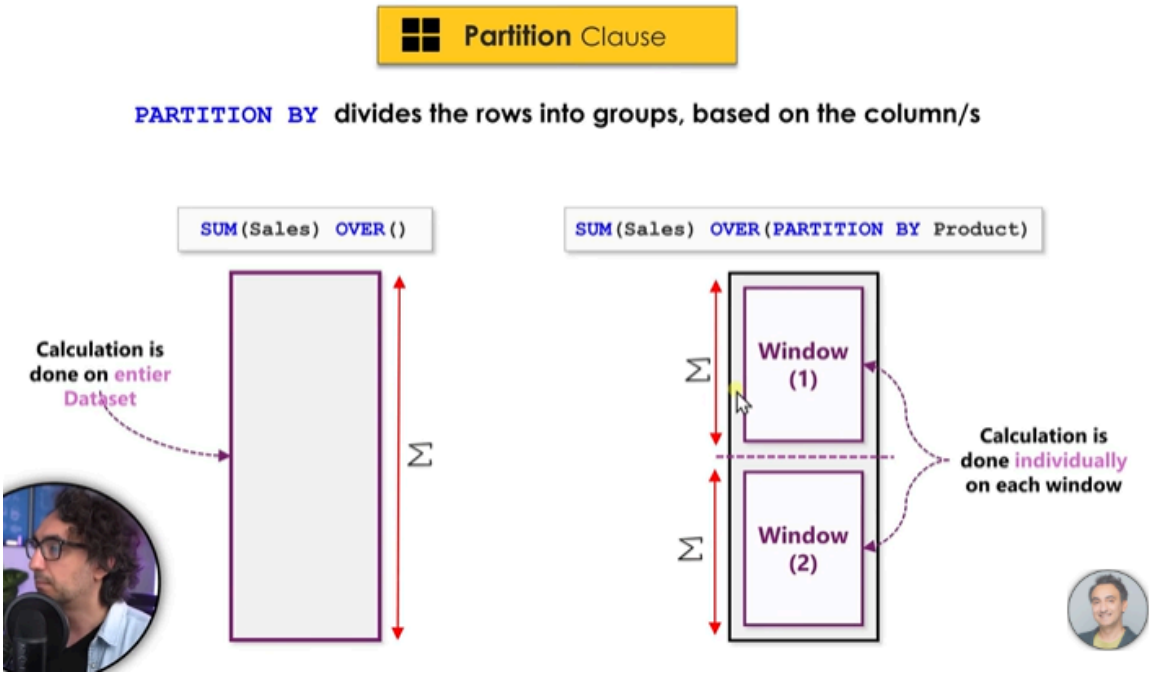
a. Window Functions

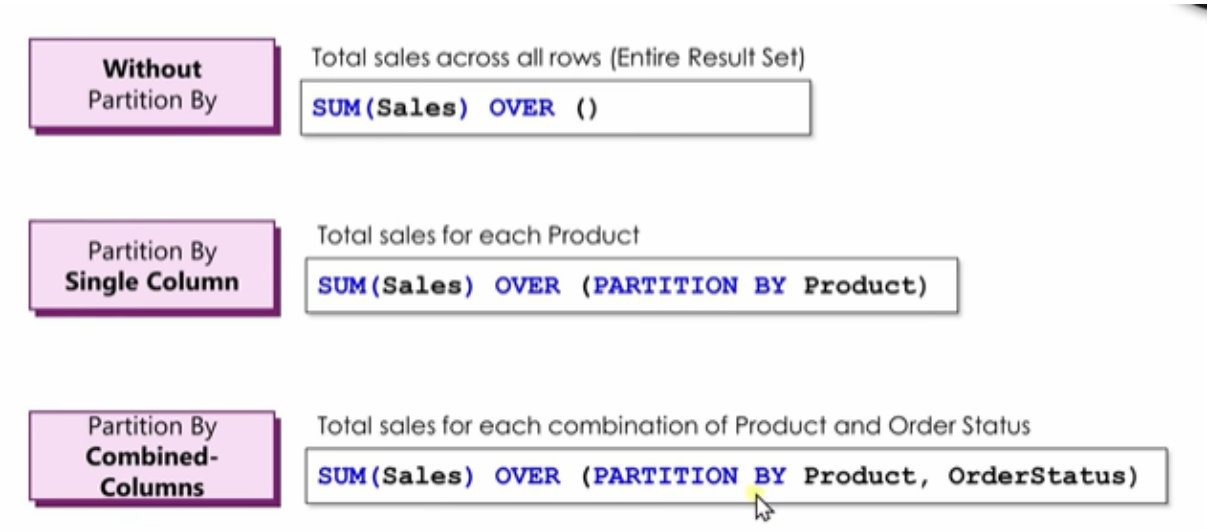


1. COUNT takes any data-type column as arugument , sum,avg,min,max tlaes only numeric columns as input parameter for that function
2. row_number,rank,.. takes no parameter
3. Value functions takes any data-type as parameters

b. Over Clause

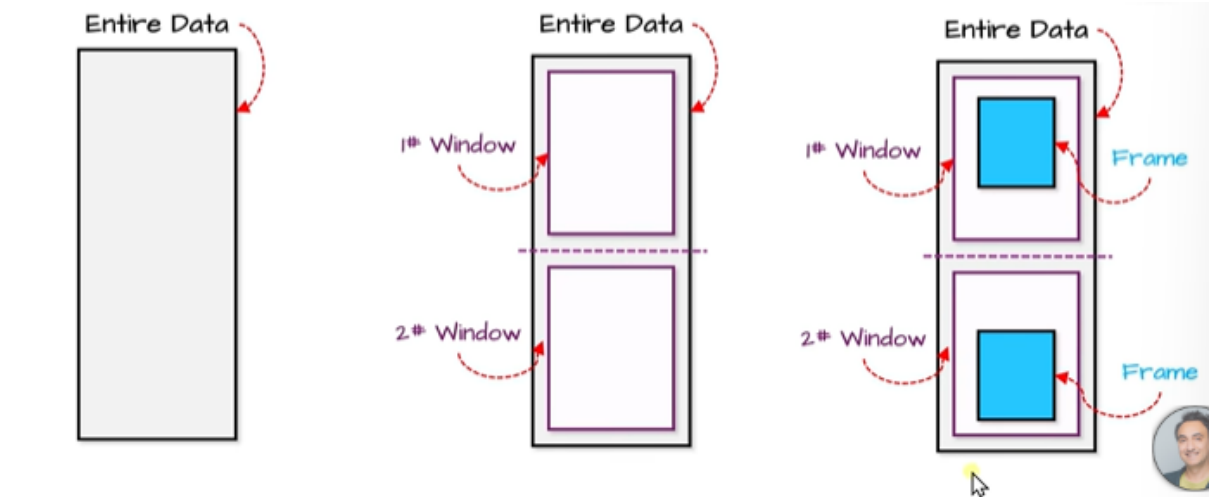
1. This says we are using Window Functions . Helps to define the window
2. PARTITION CLAUSE

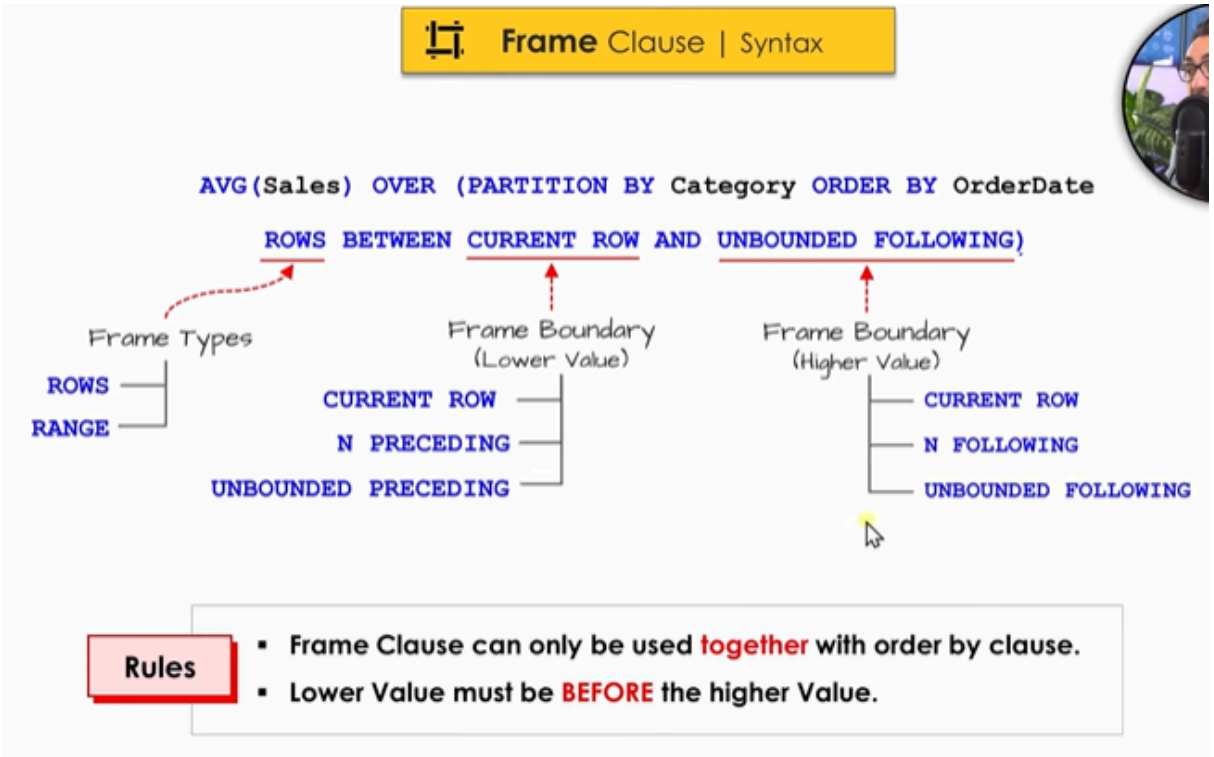




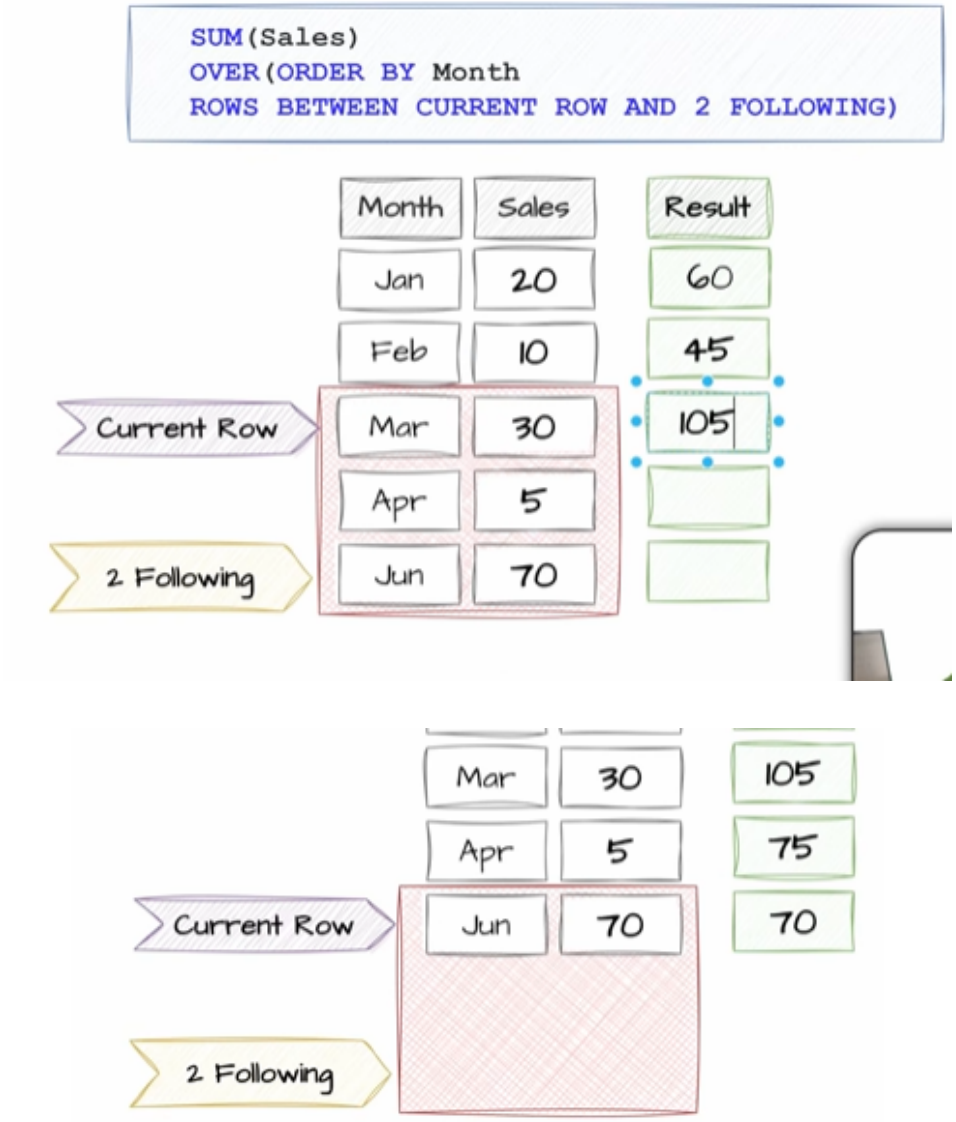
3. ORDER-BY
    a. Sorts the data within a window
    b. For Aggregate Functions this is Optional, but for RANK,VALUE Functions this is required

4. WINDOW FRAME
    a. Defines a subset of rows within each window that is relevant for the calculation





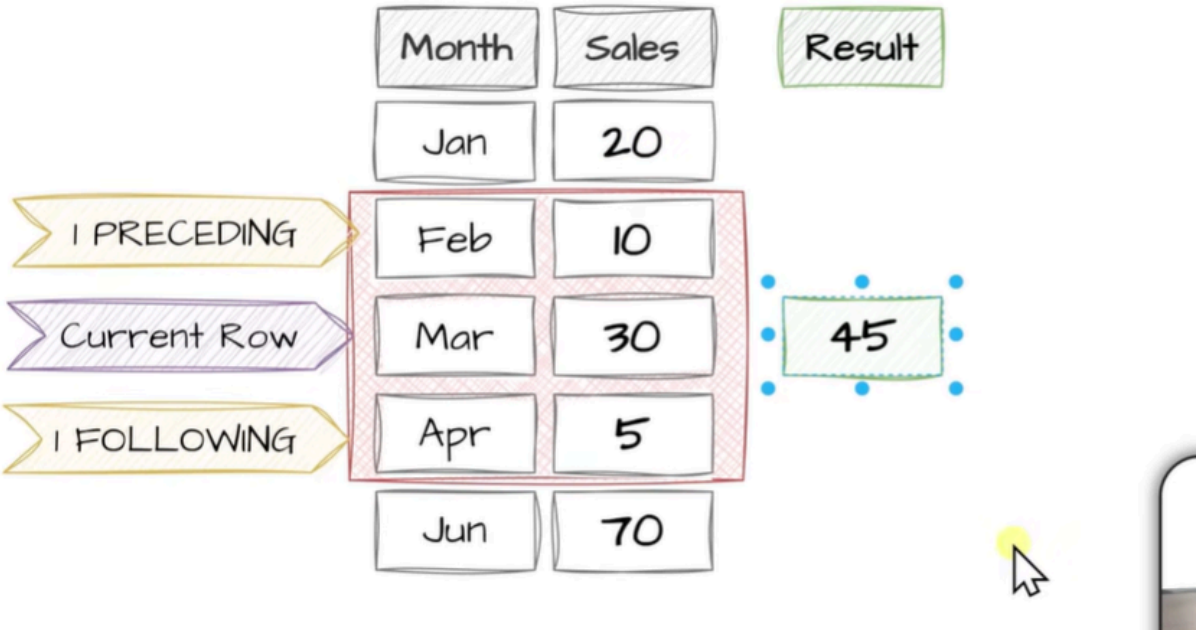
b. Helps to get Inner window → Moving Window of the size we mentioned



c. UNBOUNDED PRECEEDING → says first row of the window (fixed and donot move , not like others do)

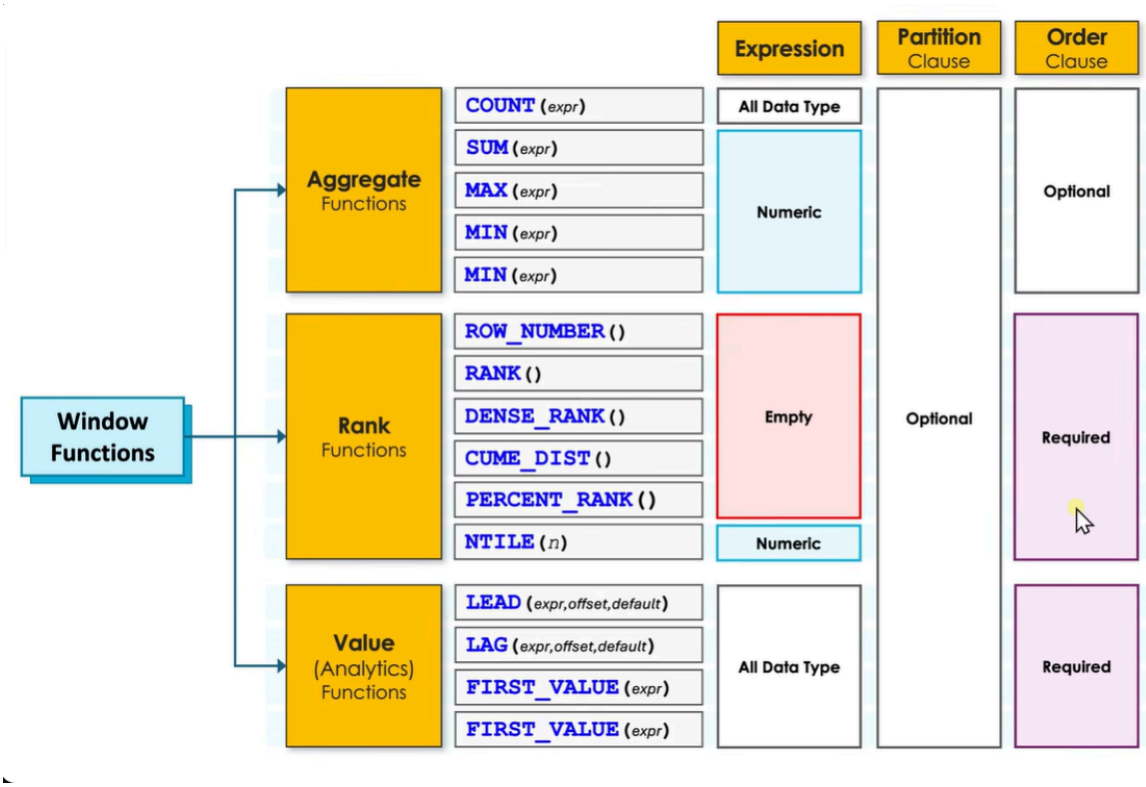
d. UNBOUNDED FOLLOWING→ says Last row of the window (fixed and donot move)

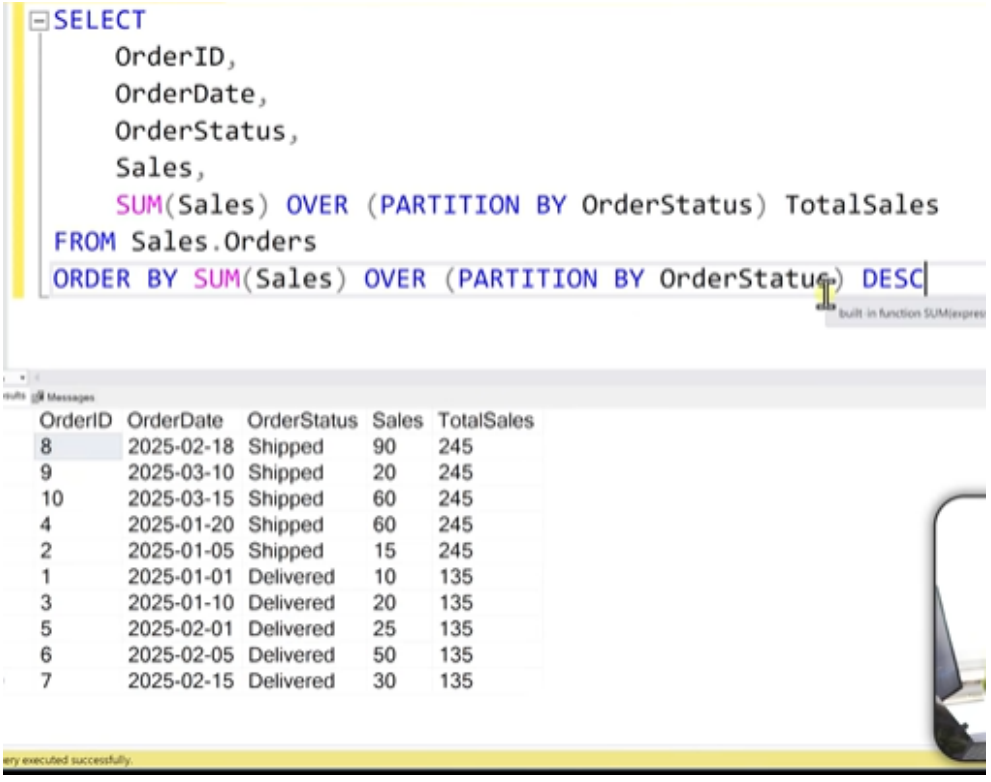
```
SUM(Sales)
OVER(ORDER BY Month
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
```



c. Rules

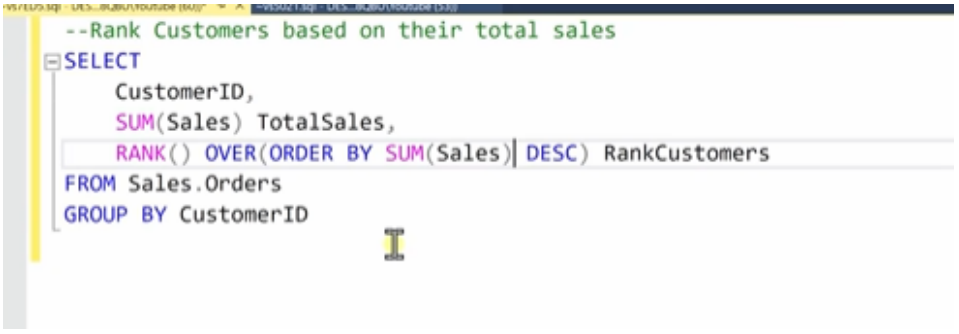
1. ORDER BY always uses a FRAME
2. Window Functions can be used only in SELECT and ORDER BY Clauses





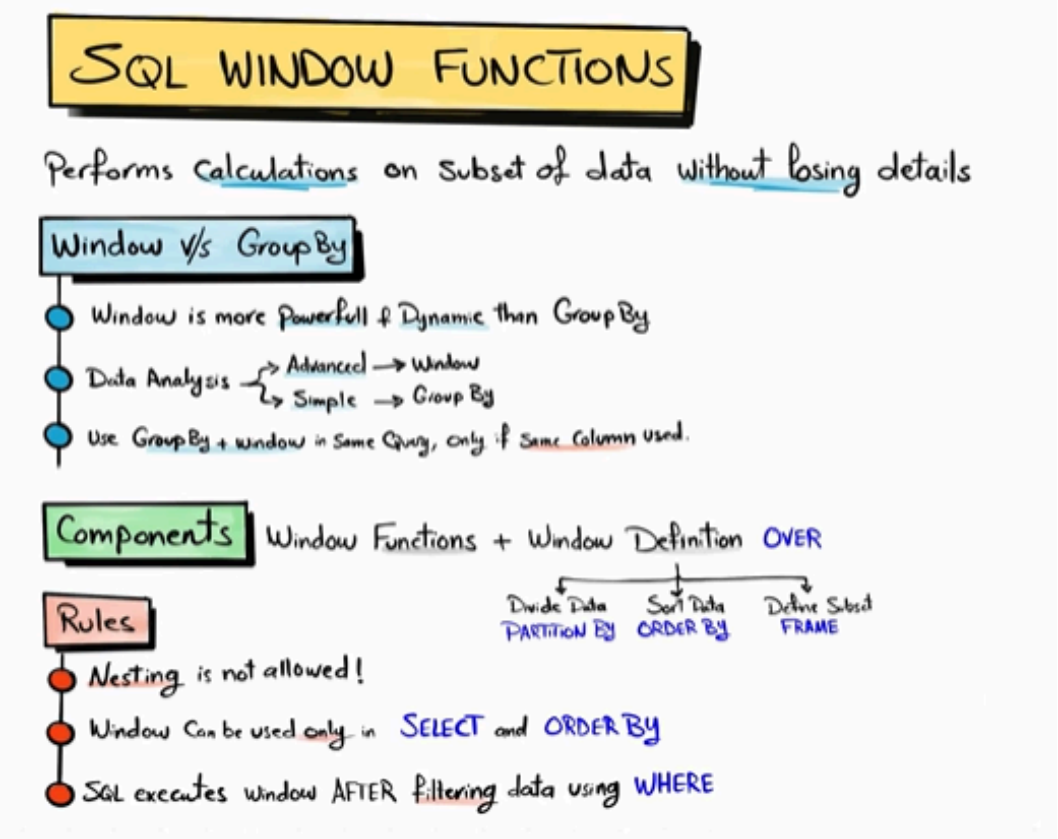
3. They are not used for FILTERING the data → Like
    a. WHERE SUM(Sales) OVER (PARTITION BY Orderstatus) >100 ⇒ not allowed

4. They are not used GROUPBY SUM(Sales) OVER() ⇒ not allowed
5. Nesting Window Function is not allowed
6. SQL execute WINDOW Functions after WHERE Clause ⇒ so they make windows for the filtered data
7. **** Window Function can be used together with GROUP BY in the same query ,only if the same columns are used
    a. We know that when GROUP-BY is used then only Aggregated Function column/the column written along with group-by can only be selected..
    b. so if we want to use WINDOW Functions along with GROUP-BY , then in WINDOW function also we should write those columns(above point read) only



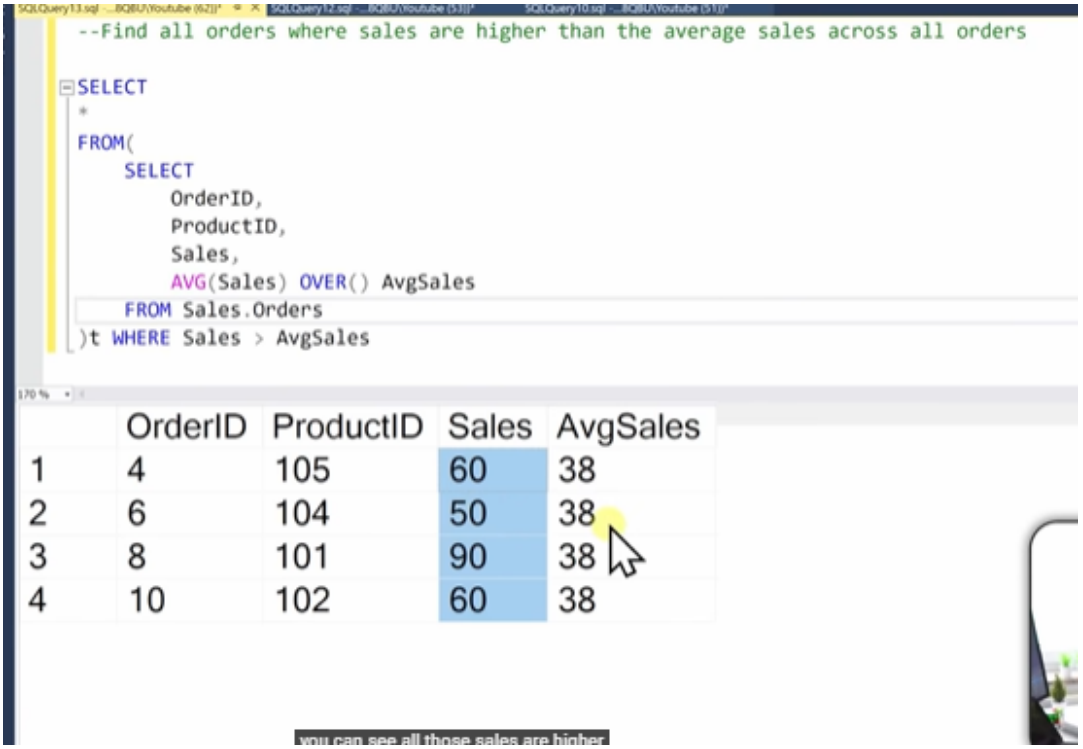
c. in place of SUM(sales) we can write CustomerID only , no other column is allowed

2. Summary



4. Window Aggregate Functions

1. COUNT
    a. Takes all data types ,
    b. Partition-by, Order-By , Frames are optional
    c. COUNT(*) count no.of rows → includes the null
    d. COUNT(column_name) counts only non-null values
2. SUM
    a. Takes only numeric data-type
    b. sums all the values in that window
    c. considers NULL as 0
3. AVG
    a. takes only numeric data-type
    b. average all the values in that window
    c. Careful with NULL



a. as we cannot use WHERE Clause for Window function , we did by Nested-Query

4. MIN/MAX
    a. returns the min,max of that particular window

a. Running and Rolling Total

1. Applications of these methods
    a. Tracking → Tracking current sales with Target Sales
    b. Trend Analysis → Providing insights into historic patterns
2. actual def

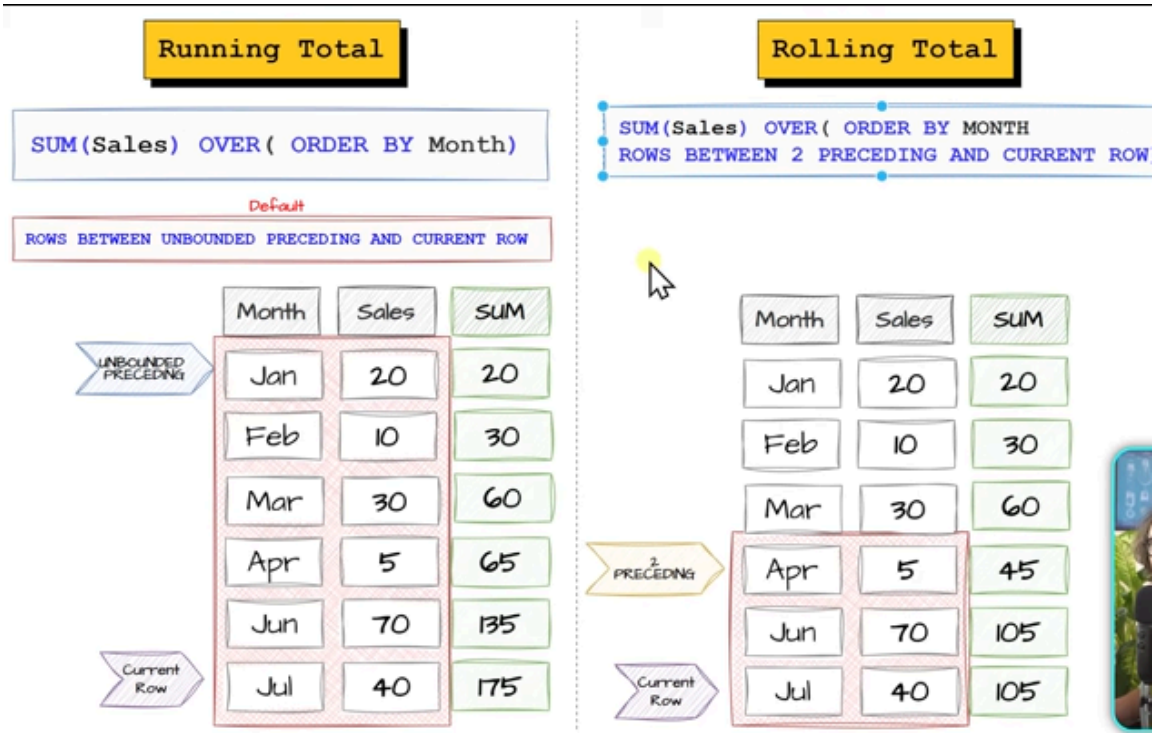
RUNNING TOTAL

Aggregate all values from the beginning up to the current point without dropping off older data.

ROLLING TOTAL

Aggregate all values within a fixed time window (e.g. 30 days).
As new data is added, the oldest data point will be dropped.

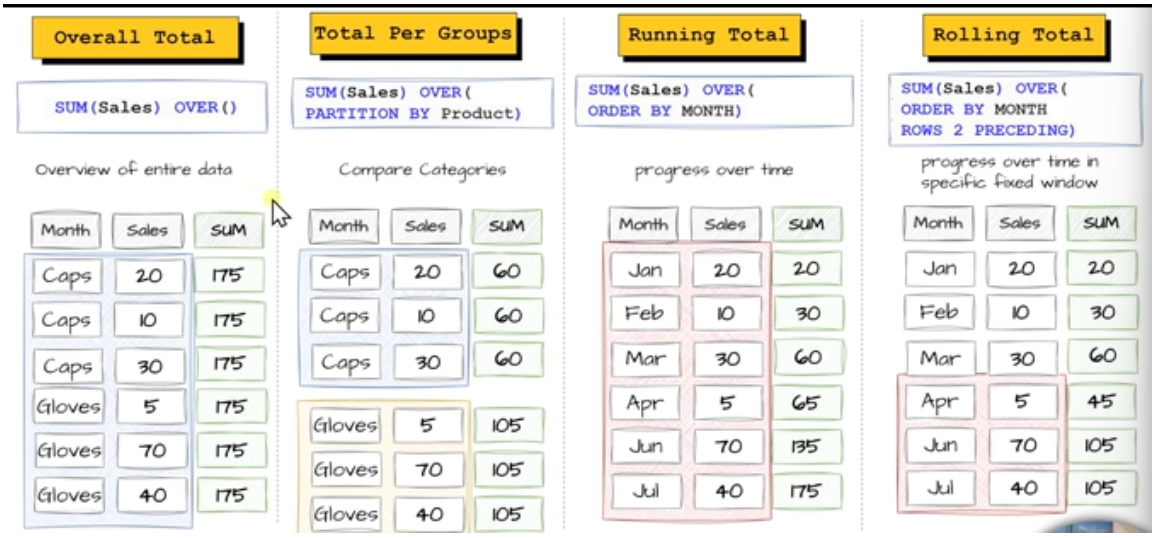
3. Actual difference



Running/moving → runs one by one without drop

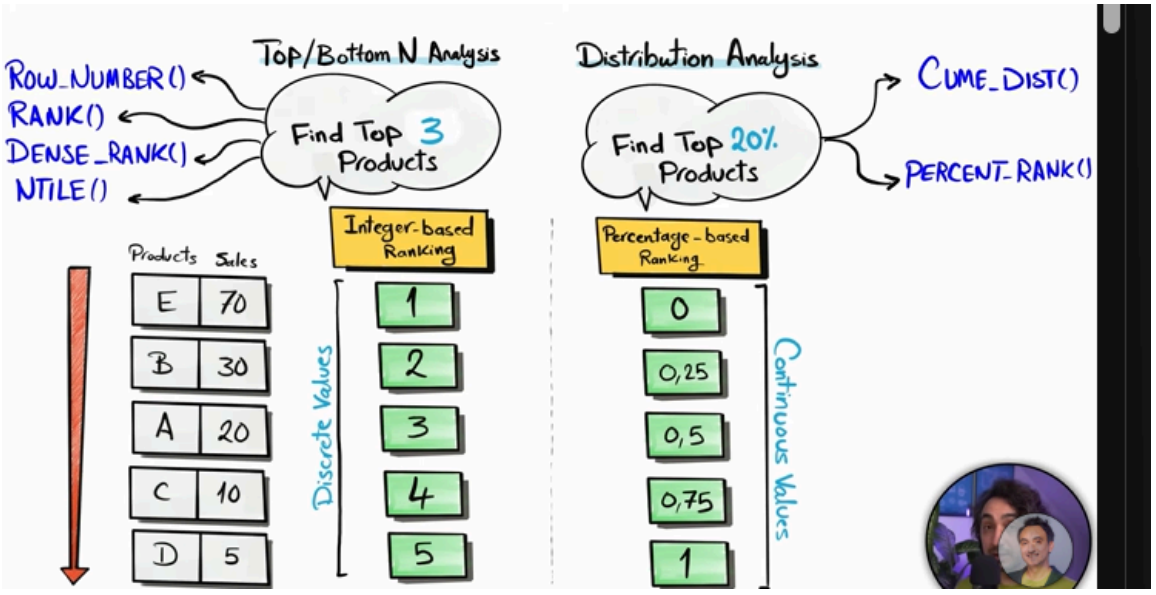
Rolling → taked the window only

4. Different ways

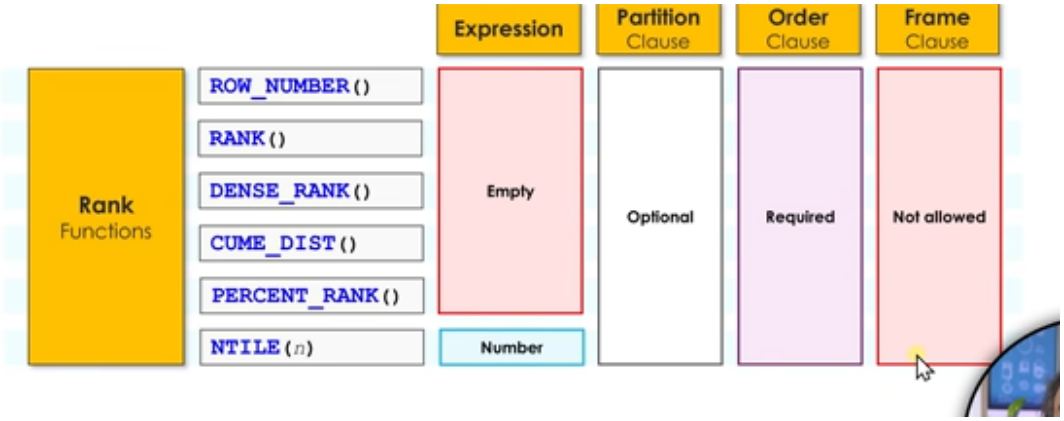


5. Window Ranking Function

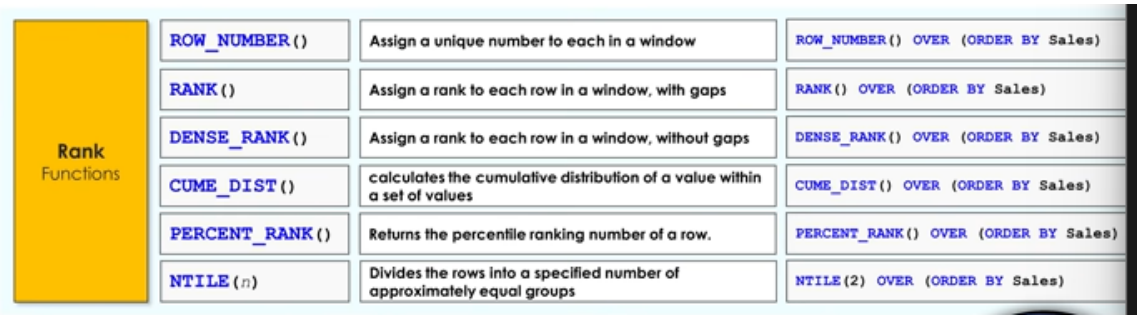
1. There are Two types of ranking
   a. Integer-based Ranking → we have Row_number(), Rank(), Dense_rank(), Ntile()
   b. Percentage-based Ranking → Cummmulative_dist() , Percent_rank()



2. Overview

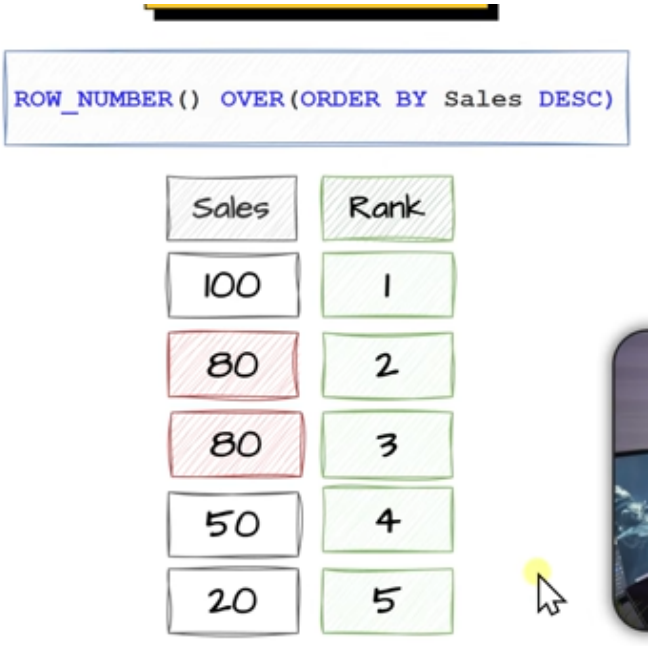


3. Syntax



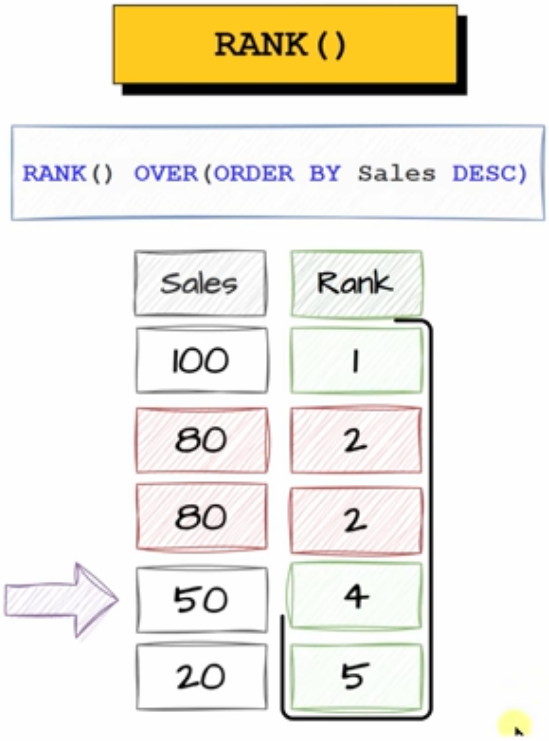
a. ROW_NUMBER()

1. Assign a unique number of each row
2. it doenot handle ties
3. Same value gets different Ranks
4. ROW_NUMBER USE CASE → TO GET UNIQUE IDs



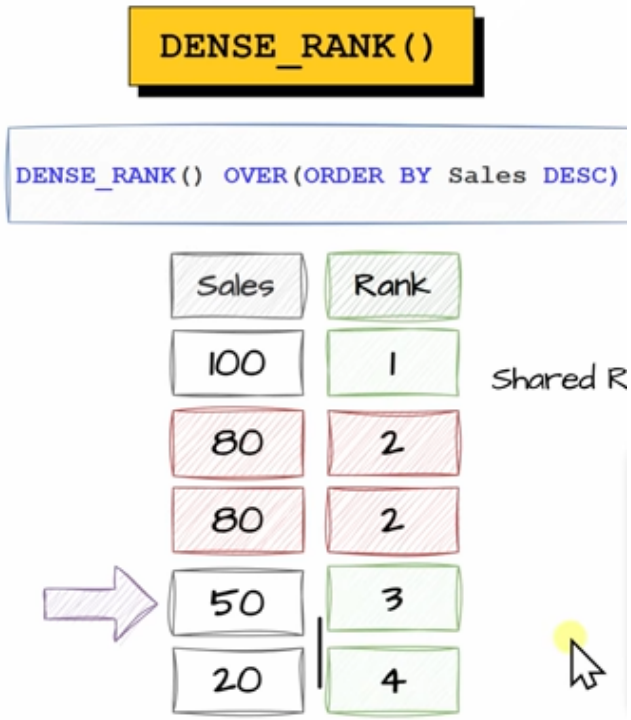
b. RANK()

1. Assign a rank to each row
2. It handles ties
3. It leaves gaps in ranking

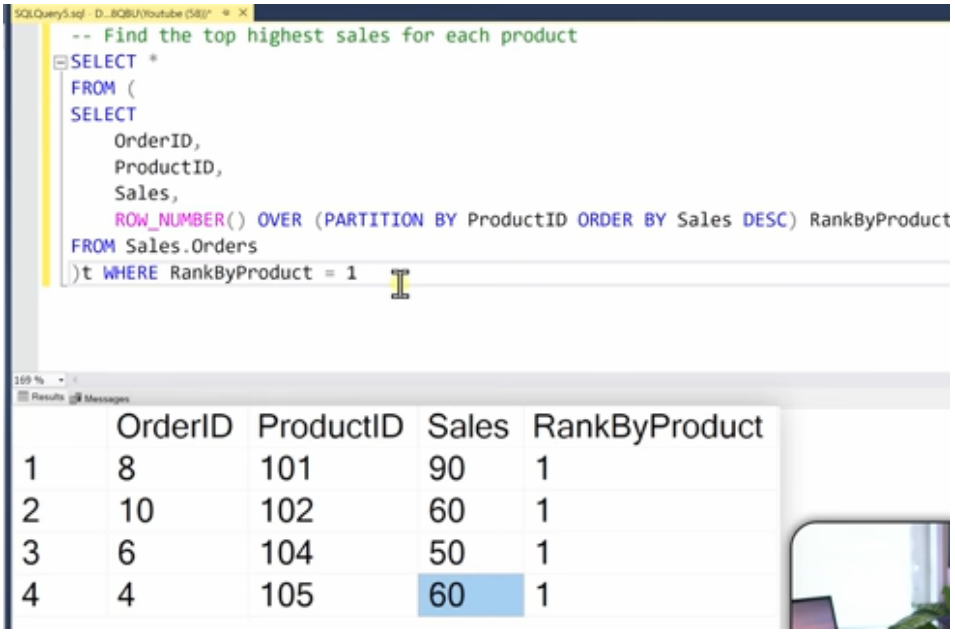


c. DENSE_RANK()

1. Assign a rank to each row
2. It handles ties
3. It doesnot leave gaps in ranking



d. ROW_NUMBER USE CASE → TOP N ANALYSIS



```
-- Find the top highest sales for each product
SELECT *
FROM (
SELECT
    OrderID,
    ProductID,
    Sales,
    ROW_NUMBER() OVER (PARTITION BY ProductID ORDER BY Sales DESC) RankByProduct
FROM Sales.Orders
)t WHERE RankByProduct = 1
```

	OrderID	ProductID	Sales	RankByProduct
1	8	101	90	1
2	10	102	60	1
3	6	104	50	1
4	4	105	60	1

e. ROW_NUMBER USE CAS E → BOTTOM N ANALYSIS

```sql
-- Find the lowest 2 customers based on their total sales
SELECT *
FROM (
SELECT
    CustomerID,
    SUM(Sales) TotalSales,
    ROW_NUMBER() OVER (ORDER BY SUM(Sales)) RankCustomers
FROM Sales.Orders
GROUP BY
CustomerID
)t WHERE RankCustomers <= 2
```

CustomerID	TotalSales	RankCustomers
2	55	1
4	90	2

1. When used Group-by make sure that the same aggregated function and it column should be write in OVER() of window function

f. ROW_NUMBER USE CASE → TO IDENTIFY DUPLICATES

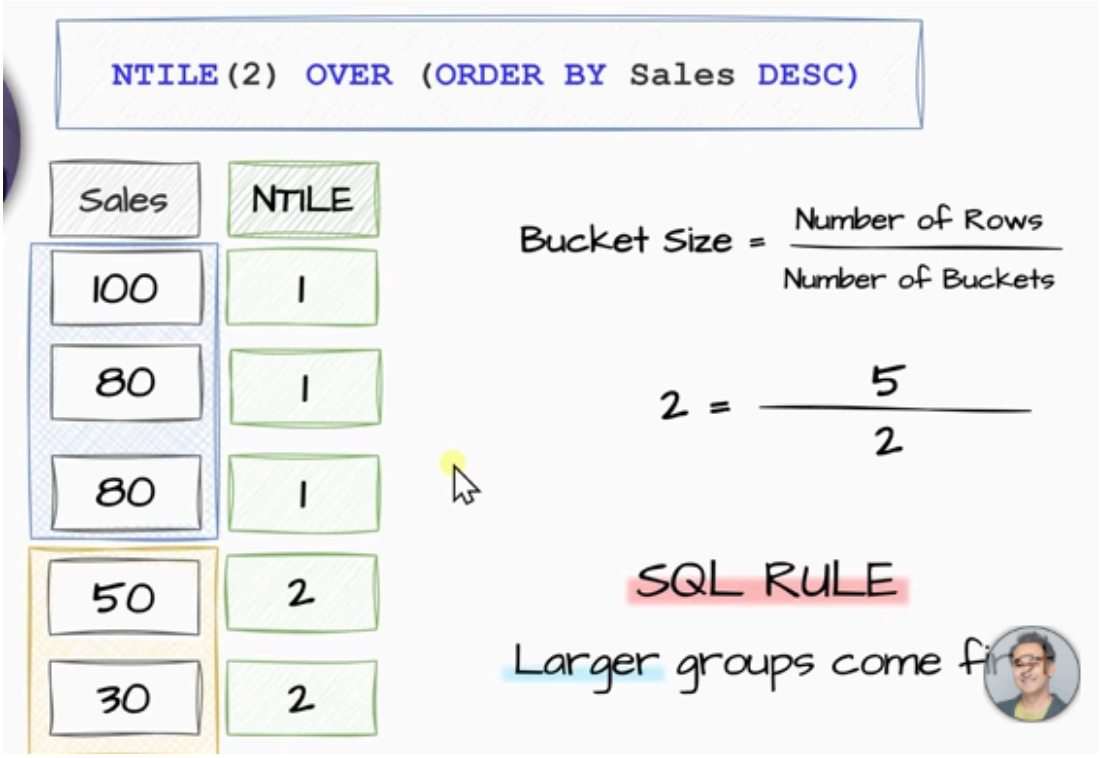
```sql
-- Identify duplicate rows in the table 'Orders Archive'
-- and return a clean result without any duplicates
SELECT * FROM (
SELECT
ROW_NUMBER() OVER (PARTITION BY OrderID ORDER BY CreationTime DESC) rn,
*
FROM Sales.OrdersArchive
)t WHERE rn>1
```

rn	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddres
2	4	105	1	3	2024-04-20	2024-04-25	Shipped	987 Victory
2	6	104	3	5	2024-05-05	2024-05-10	Delivered	543 Belmor
3	6	104	3	5	2024-05-05	2024-05-10	Delivered	543 Belmor

1. In this way the same id's will be grouped and then numbering will be assigned . so if any number>1 then bu the same id we have multiple data

g. NTILE()

1. Divides then rows into a specified number of approximately equal groups

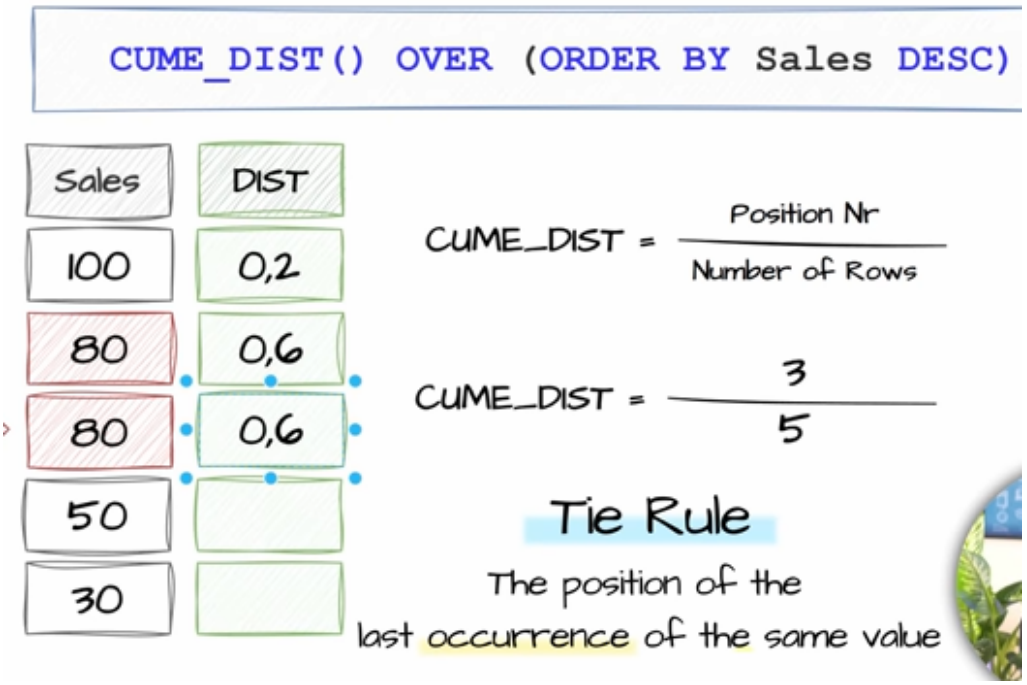


2. Use Case Example

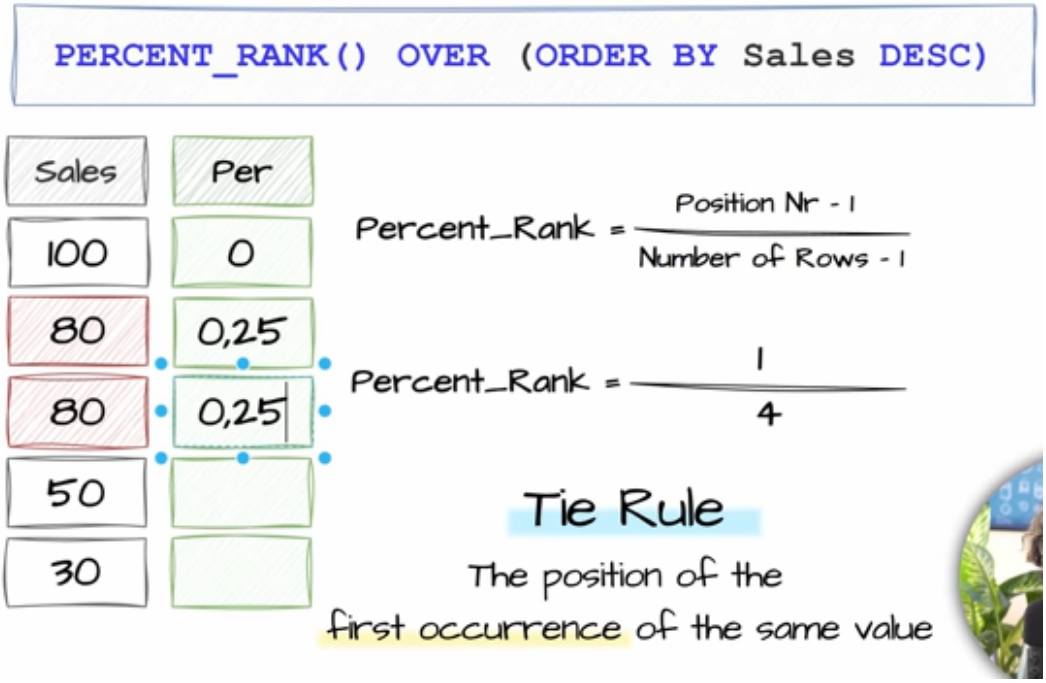
```sql
SELECT
*,
CASE WHEN Buckets = 1 THEN 'High'
     WHEN Buckets = 2 THEN 'Medium'
     WHEN Buckets = 3 THEN 'Low'
END SalesSegmentations
FROM (
    SELECT
        OrderID,
        Sales,
        NTILE(3) OVER (ORDER BY Sales DESC) Buckets
    FROM Sales.Orders
)t
```

	OrderID	Sales	Buckets	SalesSegmentations
1	8	90	1	High
2	4	60	1	High
3	10	60	1	High
4	6	50	1	High
5	7	30	2	Medium
6	5	25	2	Medium
7	9	20	2	Medium
8	3	20	3	Low
9	2	15	3	Low
10	1	10	3	Low

h. CUME_DIST()



i. PERCENT_RANK()

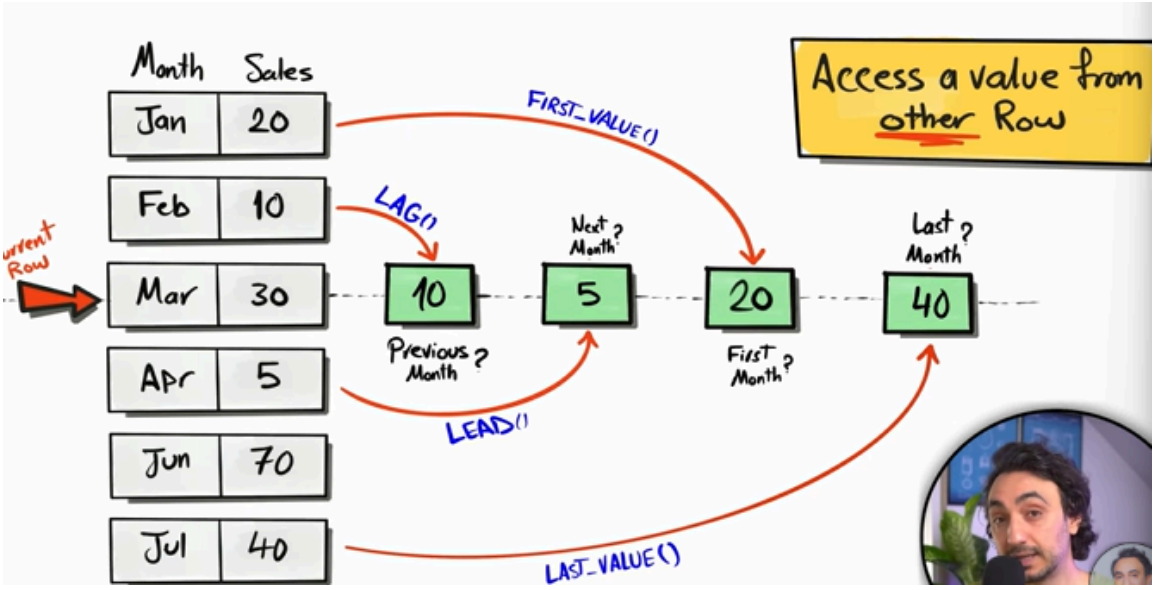


```sql
-- Find the products that fall within the highest 40% of the prices
SELECT
*,
CONCAT(DistRank * 100, '%') DistRankPerc
FROM (
    SELECT
        Product,
        Price,
        PERCENT_RANK() OVER (ORDER BY Price DESC) DistRank
    FROM Sales.Products
    )t
WHERE DistRank <= 0.4
```

Product	Price	DistRank	DistRankPerc
Gloves	30	0	0%
Caps	25	0,25	25%

6. WINDOW VALUE FUNCTION

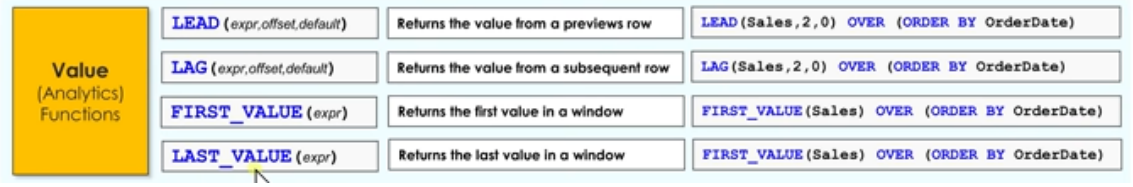
1. Deals with the Values



2. overview

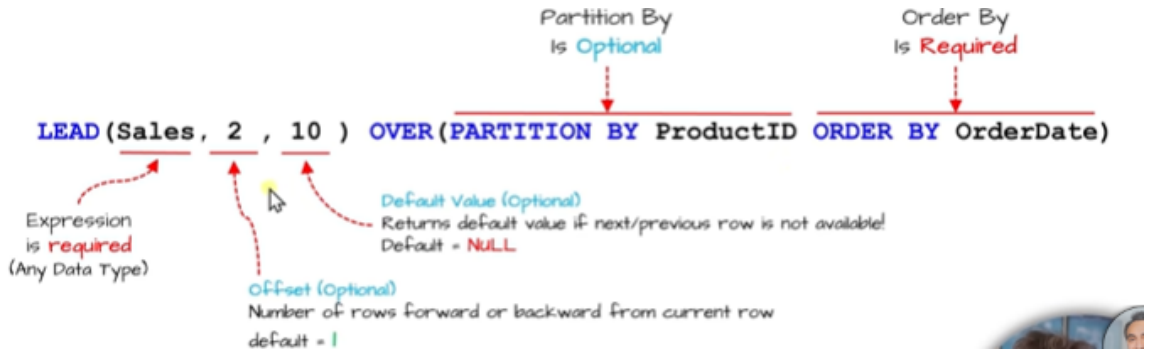


3. syntax



a. LEAD , LAG

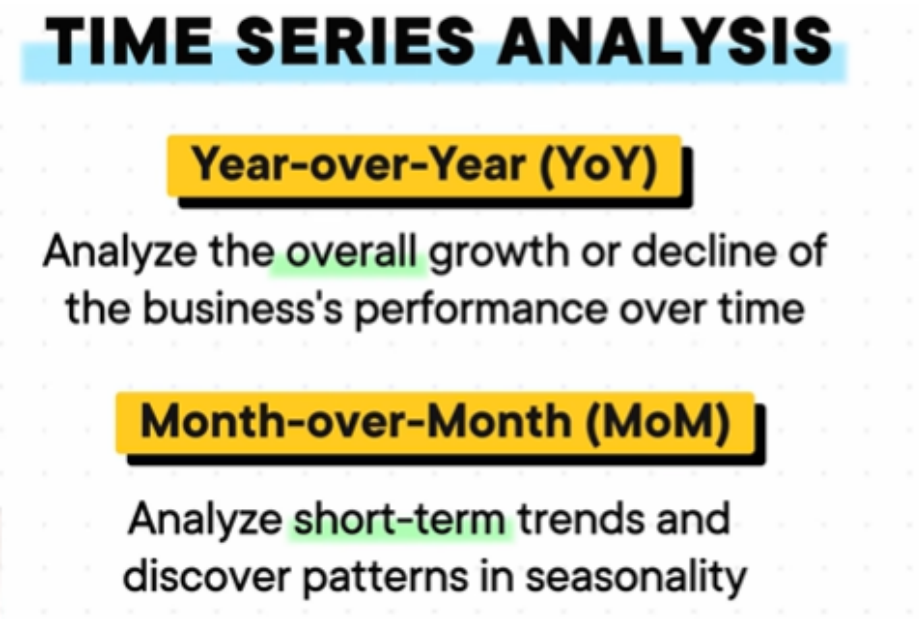
1. LEAD() → Access a value from the next row wthin a window



2. LAG() → Access a value from the previous row within a window



b. LEAD,LAG → time series analysis



```sql
-- Analyze the month-over-month performance by finding the percentage change
-- in sales between the current and previous months
SELECT
*,
CurrentMonthSales - PreviousMonthSales AS MoM_Change,
ROUND(CAST((CurrentMonthSales - PreviousMonthSales) AS FLOAT)/PreviousMonthSales *100 , 1) AS MoM_Perc
FROM (
SELECT
    MONTH(OrderDate) OrderMonth,
    SUM(Sales) CurrentMonthSales,
    LAG(SUM(Sales)) OVER (ORDER BY MONTH(OrderDate)) PreviousMonthSales
FROM Sales.Orders
GROUP BY
    MONTH(OrderDate)
)t
```

OrderMonth	CurrentMonthSales	PreviousMonthSales	MoM_Change	MoM_Perc
1	105	NULL	NULL	NULL
2	195	105	90	85,7
3	80	195	-115	-59



c. LEAD,LAG → Customer analysis

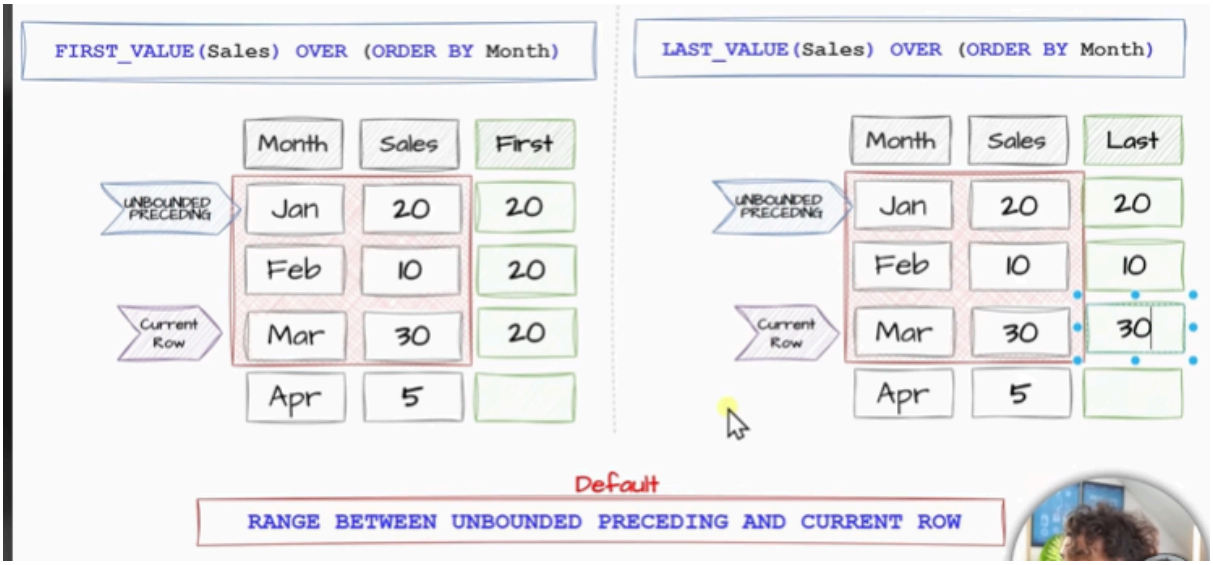
```sql
-- In order to analyze customer loyalty,
-- rank customers based on the average days between their orders
SELECT
OrderID,
CustomerID,
OrderDate CurrentOrder,
LEAD(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate) NextOrder,
DATEDIFF(day,OrderDate, LEAD(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate)) DaysUntilNextOrder
FROM Sales.Orders
ORDER BY CustomerID, OrderDate
```

OrderID	CustomerID	CurrentOrder	NextOrder	DaysUntilNextOrder
3	1	2025-01-10	2025-01-20	10
4	1	2025-01-20	2025-02-15	26
7	1	2025-02-15	NULL	NULL
1	2	2025-01-01	2025-02-01	31
5	2	2025-02-01	2025-03-10	37
9	2	2025-03-10	NULL	NULL
2	3	2025-01-05	2025-02-05	31
6	3	2025-02-05	2025-03-15	38
10	3	2025-03-15	NULL	NULL
8	4	2025-02-18	NULL	NULL



d. FIRST_VALUE, LAST_VALUE

1. FIRST_VALUE → Access a value from the first row within a window
2. LAST_VALUE → Access a value from thr last row within a window

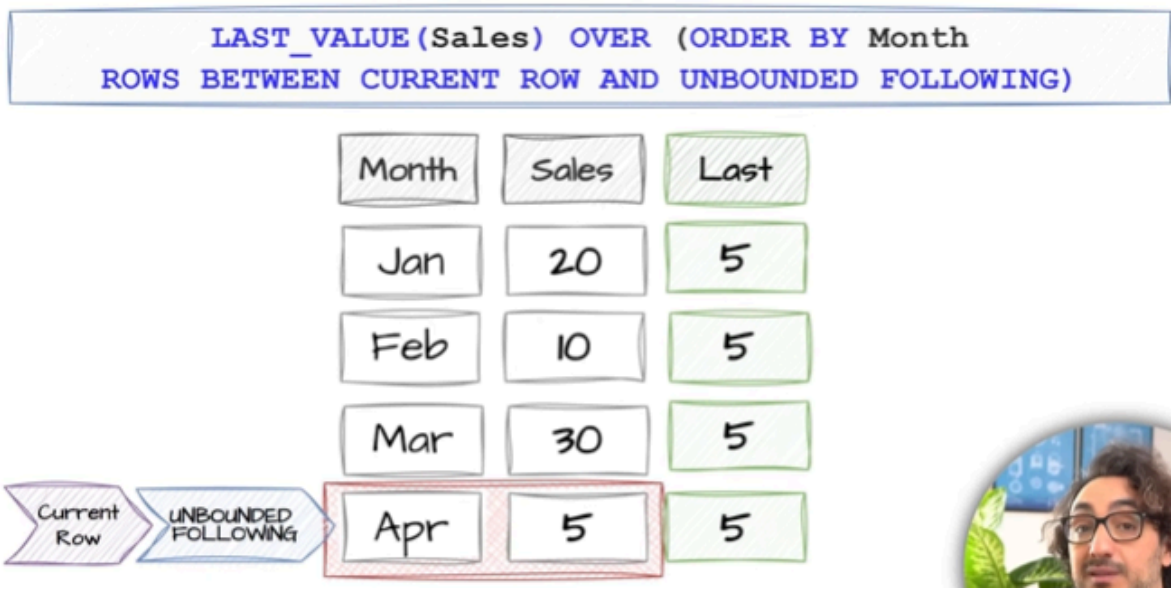


a. When we use ORDER BY then by default the RANGE will be from starting row to current row

b. so for first value → in every window for default frame the first value is the first value of table

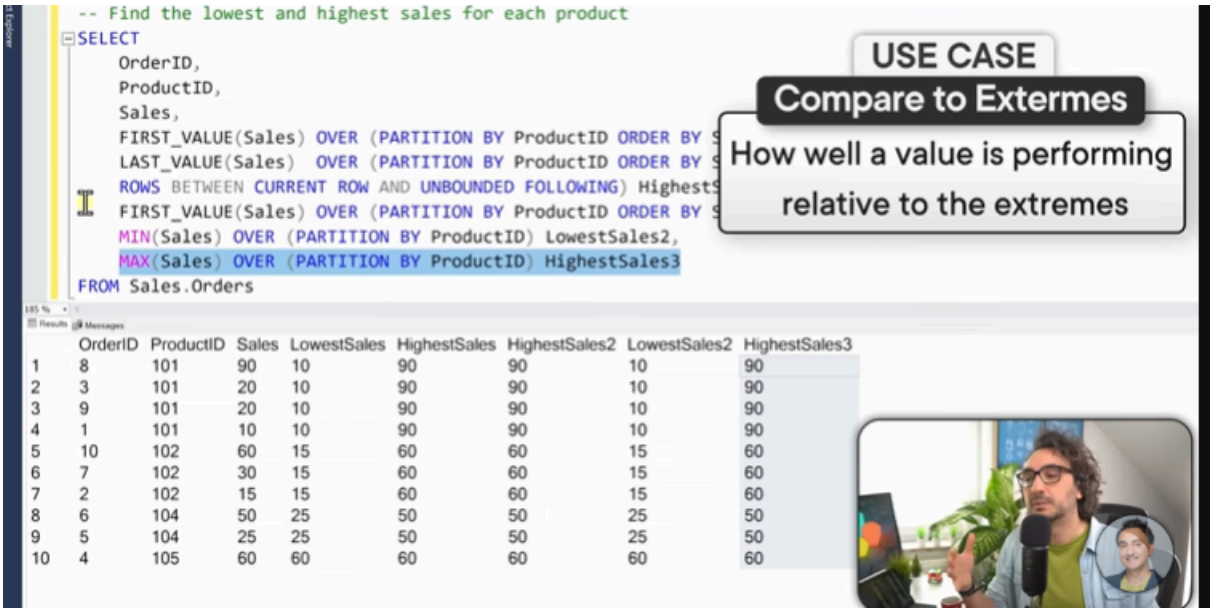
c. for Last_value → for every window for default frame the last value is the current_row only

TO GET LAST VALUE OF THE TABLE



a. Here we kept frame in such a way that the last value is always the last value of the table → because used UNBOUNDED FOLLOWING

e. FIRST_VALUE, LAST_VALUE → use case



	OrderID	ProductID	Sales	LowestSales	HighestSales	HighestSales2	LowestSales2	HighestSales3
1	8	101	90	10	90	90	10	90
2	3	101	20	10	90	90	10	90
3	9	101	20	10	90	90	10	90
4	1	101	10	10	90	90	10	90
5	10	102	60	15	60	60	15	60
6	7	102	30	15	60	60	15	60
7	2	102	15	15	60	60	15	60
8	6	104	50	25	50	50	25	50
9	5	104	25	25	50	50	25	50
10	4	105	60	60	60	60	60	60